

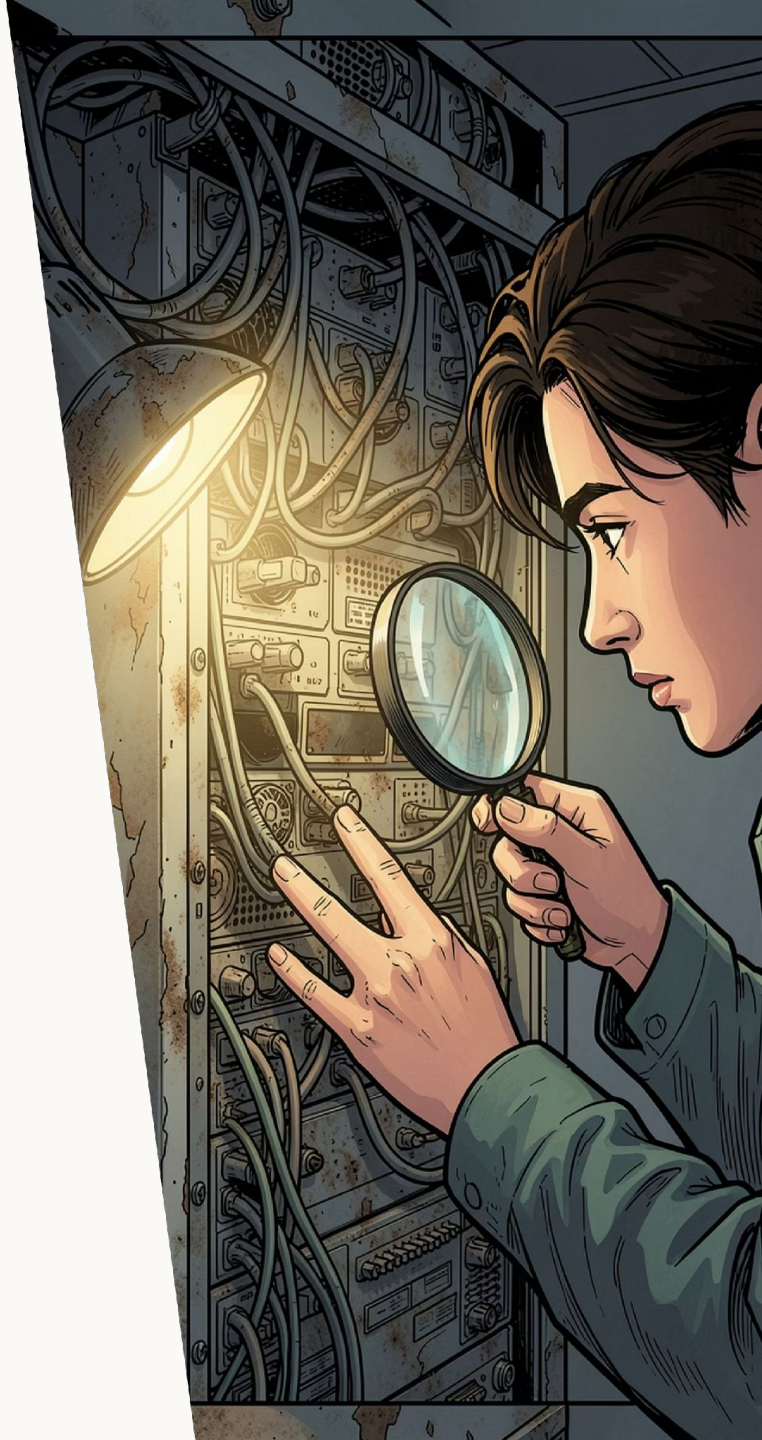
The Mythos Discovery: Cracking OpenBSD's 27-Year-Old Secret

A critical vulnerability hidden in OpenBSD's TCP stack for nearly three decades — exposed by AI-assisted analysis in 2026.

Date June 1, 2026	Found March 2026	Presented By Daxton Harris
edX Username DaxKHarris	GitHub DaxKHarris	

What This Is About

In March 2026, an AI model found a vulnerability in OpenBSD's TCP stack that had survived 27 years of expert audits, fuzz testing, and hardening efforts. To understand why that matters, you need to understand what the bug actually was — and why human reviewers kept missing it.



The Problem: TCP's Retransmission Model

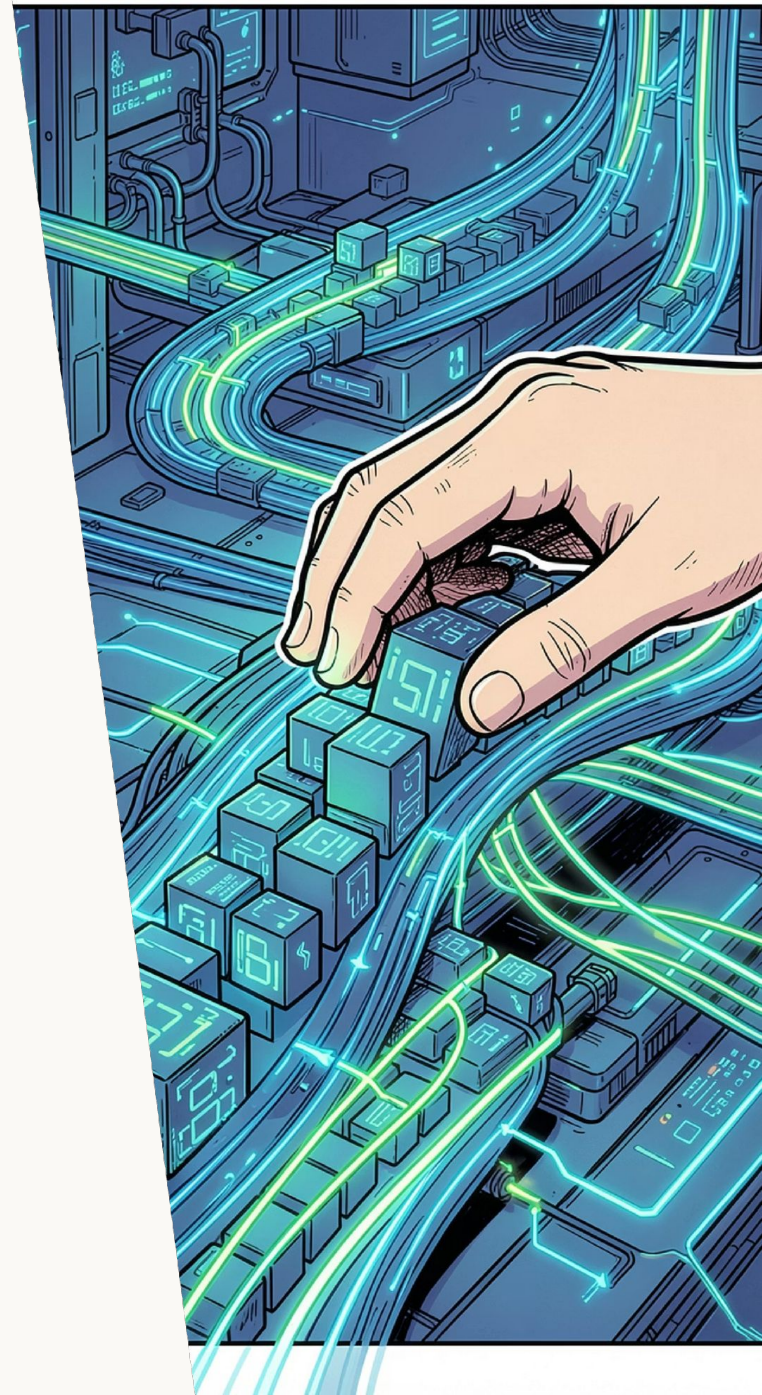
TCP's original retransmission model was blunt: lose any packet, retransmit everything from that point forward. Drop packet 4 of 10, and packets 5–10 go again too – regardless of whether they arrived successfully.



In other words, one missing packet could trigger a cascade of unnecessary resend traffic, even for data that had already reached the receiver.

The Solution: SACK

Selective Acknowledgment let receivers report exactly which blocks arrived, so senders only retransmit what's actually missing. OpenBSD implemented SACK in 1998, inheriting one silent assumption: peers send valid ranges.





Bug 1: Missing Lower Bound Validation

The kernel validated SACK ranges against the upper bound of the send window – but never the lower bound.

Upper Bound ✓

Block end must fall within the send window

Lower Bound ✗

Block start is never checked against `snd_una`

A SACK block whose start falls below the current send window is logically impossible under normal operation. The code simply never expected it.

The Exploit: Signed Integer Overflow

The kernel compared sequence numbers using `(int)(a - b) < 0`. That works only when `a` and `b` stay within 2^{31} of each other — which normal sequence numbers do. But because the lower bound is never checked, an attacker can place `sack.start` exactly 2^{31} away from `snd_una`. The subtraction overflows, the signed cast flips negative, and the comparison passes.

In other words: a logically impossible range becomes valid to the kernel because the sign changes at the overflow boundary.

2 = 30

2 = 31



2 = 31



Why That's Not Enough to Crash Anything — Yet

A corrupted hole entry in the linked list is bad, but it doesn't immediately crash the kernel. It just puts the list in a state the code never anticipated. That's where Bug 2 comes in — and Bug 1 is precisely what makes Bug 2 reachable.

Bug 1

Corrupts the hole entry in the linked list.

Immediate impact

Unsafe state, but no crash on its own.

Why it matters

It creates the exact condition Bug 2 needs.

Bug 2: Null Pointer Dereference

The kernel tracks unacknowledged data as a singly linked list of holes. Two rules evaluate sequentially on each SACK block:

Deletion

If this block covers an existing hole, free it.

Append

If this block extends past the list end, attach a new hole.

The malicious range — made valid by Bug 1 — is crafted to trigger both simultaneously. Deletion frees the only node in the list, setting the tail pointer to NULL. Append then immediately executes `tail->next = new_hole`. The kernel writes to address `0x0`. Kernel panic.

Why the Code Is Written This Way

The two rules were written as sequential checks, not mutual exclusions — because under valid inputs, both firing simultaneously is impossible.

Rule 1

Free the block if it falls inside an existing hole.

Rule 2

Append a new hole if the block extends past the list end.

A real SACK block can't simultaneously be inside a hole and past the end of the list. The missing lower bound check is what collapses that assumption.

Why Humans Missed It for 27 Years

Mythos' approach

Mythos reads source code, forms hypotheses about where vulnerabilities might exist, runs the actual software, and uses debuggers to confirm findings.

Why audits and fuzzers missed it

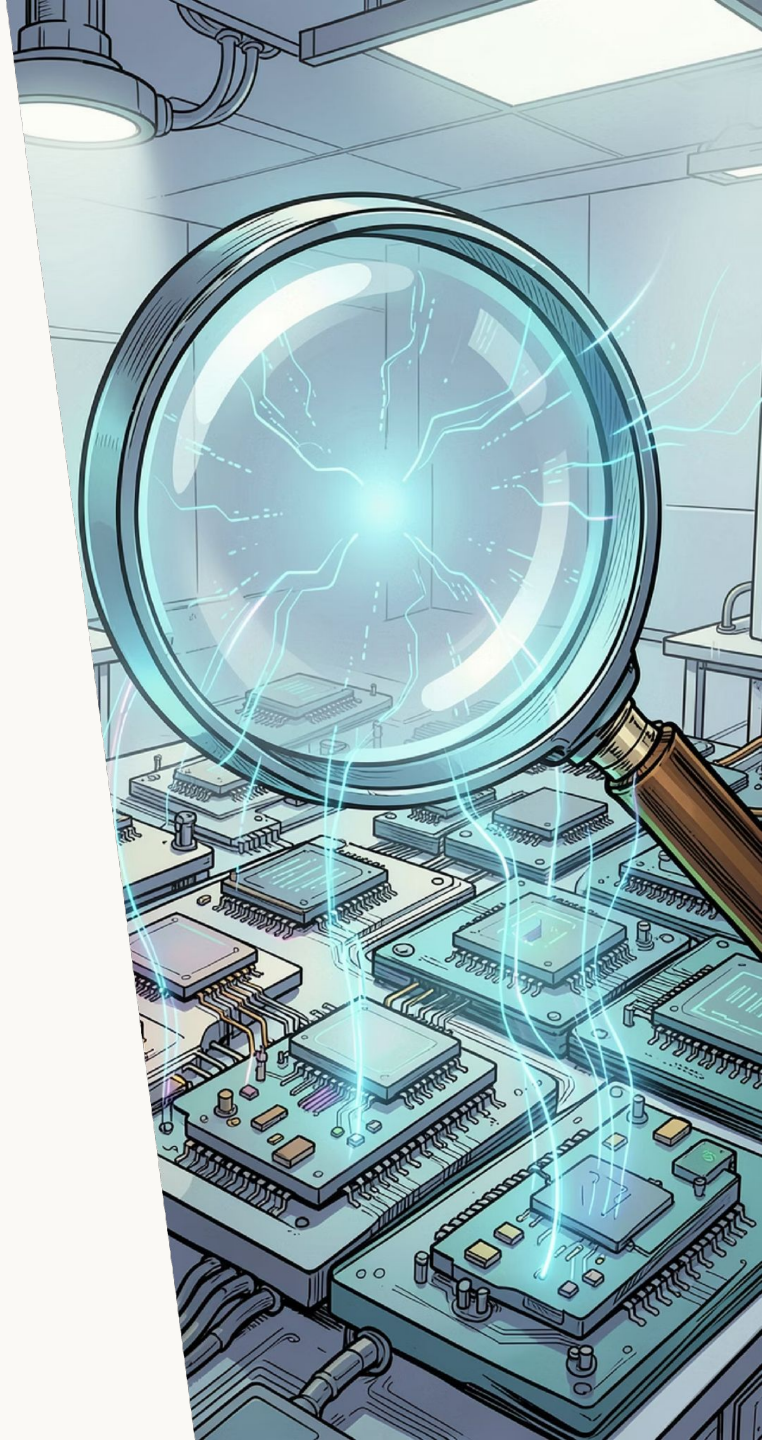
Human auditors and fuzzers didn't miss this because they were careless — they missed it because the reachability of Bug 2 requires first constructing the specific integer overflow condition from Bug 1. The flaw survived millions of automated fuzz tests and multiple expert audits.

Why standard fuzzing failed

Standard fuzzers generate random inputs; they don't reason about the arithmetic relationship between two separate validation gaps across a linked list traversal.

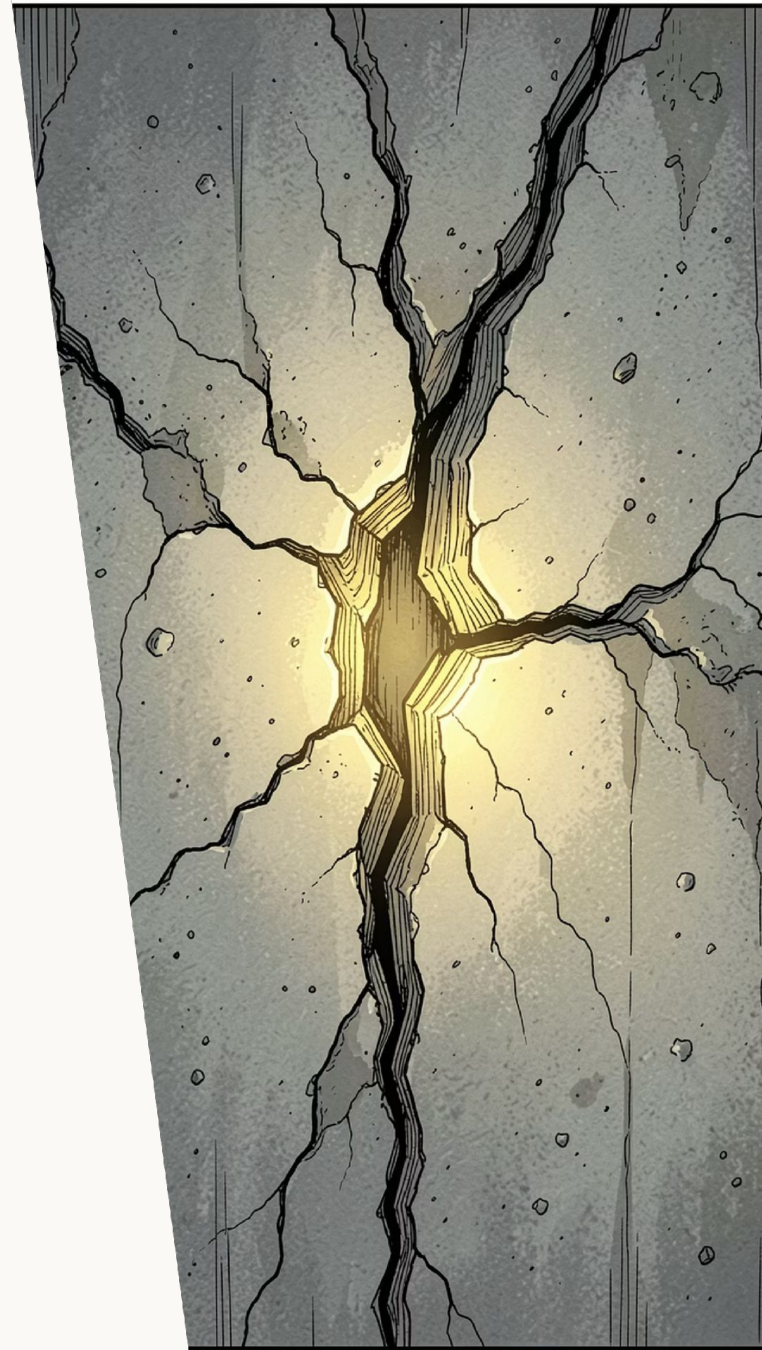
The Discovery

Claude Mythos Preview found it autonomously, in under \$50 worth of compute on a single scaffold run. It is part of Anthropic's Project Glasswing. A defensive security initiative that has so far identified over 10,000 high or critical severity vulnerabilities across major operating systems and browsers.



Impact

- Remote kernel panic — single malformed TCP packet, no authentication required
- Affects OpenBSD 7.8 and earlier
- Hidden from 1998 to 2026





Recommendations

1 Validate every bound, not just the obvious one

The upper bound check existed because it was the intuitive constraint. Lower bounds feel redundant when the RFC implies valid peers. They aren't — validate them explicitly regardless of what the protocol spec implies about sender behavior.

3 Don't trust legacy code just because it's old

OpenBSD is one of the most scrutinized codebases on earth. Age and reputation are not a substitute for systematic re-audit — particularly for protocol parsing code written before modern static analysis tooling existed.

2 Treat sequential code paths as potential simultaneous triggers

The deletion/append logic was possible because mutual exclusivity wasn't enforced. Any time two code paths share state and run sequentially, audit whether a malformed input can activate both. This is especially critical in kernel-level linked list manipulation.

4 Fuzz with semantic awareness

Standard fuzzers won't find this class of bug because they don't reason about arithmetic relationships between fields. Tools that model protocol state and integer boundary conditions — or AI-assisted code analysis — are now necessary complements to traditional fuzzing.